Executive Summary:

The objectives of the assignment are to analyze a given zip file called “SaveForPippin.zip” and determine a way to gain access to the “pippin” account. This account is included in the system we are tasked with analyzing. To complete the assignment, one must locate and recover the content of the pippinflag.txt file. While using the SSH (Secure Shell) network protocol to login, the user “pippin” may appear smart, yet he inadvertently created a vulnerability by acting with a poor cybersecurity practice known as “security by obscurity”. By hiding his SSH key through a variety of zip files and obscuring file types, he hoped to keep his SSH Key a secret. This practice imposes the risk of an attacker doing an analysis on the file, quickly realizing the file type obscurity, and reverse engineering their way to the original file containing the SSH key.

The original file that contains the SSH key also attempts to hide it within thousands of other unrelated lines, portraying another practice of “security by obscurity”. Once one gains access to “pippin”’s account using the SSH key, one can also explore the locally available file system and, within its contents, find a .csv (Comma-Separated Values) file, meaning, the simple plain text file storing tabular data containing login information, username and password, of an ongoing SMB communication protocol that currently shares a file, in this case, the flag file.

This attack shows vulnerabilities in “pippin”’s attempts to hide his SSH key and the flag file. We reinforced that security by obscurity is a bad practice, and a possible solution may have been Pippin encrypting his SSH key with a trusted algorithm and storing the encryption key in a safe place. Furthermore, usernames and passwords should not be openly stored in a CSV file with no permissions. That way, it would prevent anyone from logging in as Pippin and retrieving the flag file.

The first attack consisted in using the `file` Linux command to determine the file type of the contents of the SaveForPippin.zip file. Once we know that, we proceed to rename the file to its appropriate file extension, and we repeat this process until the original file is unearthed. From there, it became necessary to filter the contents of the original file to find out the delimiters of the hidden SSH key; that way, we can store it in a temporary file. Using that SSH key to log in as “pippin”. In the second attack, we explored the nearby files and found a .csv file containing the username and password of user “pippinflag”, as well as a local SMB (Server Message Block) service running within ports 139/445 where user Pippinflag had access to. By utilizing the stored and unsecured username password, the team was able to quickly gain access to the pippinflag.txt flag file, using `smbclient` command.

1 Objectives

1.1 Gain Entry to the Pippin Account

The team was tasked with gaining entry into the user “pippin” account present on the system. A starting point was provided to us via the “SaveForPippin.zip”.

1.2 Locate and Recover “pippinflag.txt”

Upon gaining entry to the “pippin” user account, the team should locate and recover the flag file “pippinflag.txt”.

2 Attacks

For each exploit used, fill out the following table. If one exploit depends on another to achieve its goal, reference the dependency in the table for that exploit.

2.1 SSH Key Extraction

Exploit #1	Security-Through-Obscurity SSH Key Exposure via SaveForPippin.zip
Description	This attack retrieved the “pippin” account’s OpenSSH private key hidden by misleading file extensions and layers of compression. This type of attack was not a software vulnerability but instead was credential exposure due to poor secrets management. The team through every layer found each container format, extracted its contents and repeatedly discovered a PEM-formatted key block that allowed access to the “pippin” account using SSH (Secure Shell). This exploit allowed the attacker to have full-user access to the “pippin” account and is a pre-condition to Exploit #2 which depended on the data found after logging in as “pippin”.
Objectives	Objective 1.1: Gain Entry to the Pippin Account
Assumptions	This exploit had several assumptions regarding the environment and available resources: ● Local access to the student VM (Virtual Machine) with read access to /home/student/pippin/SaveForPippin.zip ● Standard compression tools (tar, unzip,gzip,bzip2,xz) and OpenSSH client are available ● SSH on Student account accepts key-based authentication for user “pippin” If any of the above mentioned assumptions turned out to be false, the taken exploit path would become more complex or even unworkable.

Exploit #1	Security-Through-Obscurity SSH Key Exposure via SaveForPippin.zip
Findings	The attack was successful. An OpenSSH private key block was hidden by several nested archives and cover-text file. The vulnerability came from depending on security through obscurity, which allowed the placing of sensitive authentication information in a user-readable file without encryption or access controls. Using the extracted SSH login allowed access to the "pippin" account.
Mitigations	To reduce risks shown in this exploit, the adoption of the following practices recommended: ● Do not store private keys in archives users can access; the use of a secrets manager or encrypted vault would be ideal ● Limit filesystem permissions and scan secrets on a regular basis to identify keys at rest ● SSH keys should have passphrases and require hardware-mediated authentication
Tools Used	

file	Repeatedly used to identify the real format of every archive layer. Many files frequently contained the wrong extensions, ***file*** correctly identified the file type
unzip	Decompressed zip archives. Flags: -d (used to specify destination directory)
gunzip, bunzip2, unxz	These decompress single layer gzip, bzip2, and xz archives Flags: -k (used to keep original file)
grep	Searched through a loaded text file to see if the SSH key was within the file. Specifically it was used to locate the markers -----BEGIN OPENSSH PRIVATE KEY----- and -----END OPENSSH PRIVATE KEY-----
awk	Extracted the SSH key block from the larger text file that had the obscure story text
tr	Removed potential carriage returns or hidden characters in the key file, to make sure SSH would accept it
chmod	Readjusted file permissions on the recovered key to meet SSHs condition that the private keys be readable only by the file's owner.
ssh	Used the extracted key to log into the pippin account on the local host Flag

Exploit #1	Security-Through-Obscurity SSH Key Exposure via SaveForPippin.zip
Commands Used with Syntax	1. cd pippin 2. unzip SaveForPippin.zip 3. file PippinsEyesOnly.tar.gz 4. bunzip2 -k PippinsEyesOnly.tar.gz 5. file PippinsEyesOnly.tar.gz.out 6. mv PippinsEyesOnly.tar.gz.out PippinsEyesOnly.tar.gz 7. file PippinsEyesOnly.tar.gz 8. gunzip PippinsEyesOnly.tar.gz 9. file PippinsEyesOnly.tar 10. unzip PippinsEyesOnly.tar 11. file PippinsEyesOnly.xz 12. bunzip2 -k PippinsEyesOnly.xz 13. file PippinsEyesOnly.xz.out 14. mv PippinsEyesOnly.xz.out PippinsEyesOnly1.xz 15. unxz PippinsEyesOnly.xz 16. file PippinsEyesOnly 17. mv PippinsEyesOnly PippinsEyesOnly.gz 18. gunzip PippinsEyesOnly.gz 19. file PippinsEyesOnly 20. cat PippinsEyesOnly 21. grep "BEGIN OPENSSH" PippinsEyesOnly 22. grep "END OPENSSH" PippinsEyesOnly 23. awk '/-----BEGIN .*PRIVATE KEY-----/{flag=1} flag{print} /-----END .*PRIVATE KEY-----/{flag=0}' PippinsEyesOnly > pippin_key

Exploit #1	Security-Through-Obscurity SSH Key Exposure via SaveForPippin.zip
	24. tr -d '\r' < pippin_key > pippin_key.fixed && mv pippin_key.fixed pippin_key 25. chmod 600 pippin_key 26. ssh -i pippin_key pippin@localhost
Details	This exploit began with finding the provided SaveForPippin.Zip from the Ubuntu student account. Using the File command showed that the outer ZIP archive contained compressed files/ archives. Going through the layers of the archive revealed compressed or mis-labeled containers. By repeating the command file and applying the correct tools: unzip, gunzip, bunzip2, or unxz, the team was able to peel away successive layers of compression.

```
student@cs647:~$ cd pippin/
student@cs647:~/pippin$ ls
SaveForPippin.zip
student@cs647:~/pippin$ file SaveForPippin.zip
SaveForPippin.zip: Zip archive data, at least v2.0 to extract, compression method=deflate
student@cs647:~/pippin$ unzip SaveForPippin.zip
Archive:  SaveForPippin.zip
  inflating: PippinsEyesOnly.tar.gz
student@cs647:~/pippin$ file PippinsEyesOnly.tar.gz
PippinsEyesOnly.tar.gz: bzip2 compressed data, block size = 900k
```

Screenshot 1.1.1 Accessing given zip file

```
student@cs647:~/pippin$ bunzip2 -k PippinsEyesOnly.tar.gz
bunzip2: Can't guess original name for PippinsEyesOnly.tar.gz -- using PippinsEyesOnly.tar.gz.out
student@cs647:~/pippin$ ls
PippinsEyesOnly.tar.gz  PippinsEyesOnly.tar.gz.out  SaveForPippin.zip
student@cs647:~/pippin$ file PippinsEyesOnly.tar.gz.out
PippinsEyesOnly.tar.gz.out: gzip compressed data, was "PippinsEyesOnly.zip", last modified: Wed Sep  3 04:58:14 2025, from Unix, original size modulo 2^
32 987722
student@cs647:~/pippin$ mv PippinsEyesOnly.tar.gz.out PippinsEyesOnly.tar.gz
student@cs647:~/pippin$ ls
PippinsEyesOnly.tar.gz  SaveForPippin.zip
student@cs647:~/pippin$ file PippinsEyesOnly.tar.gz
PippinsEyesOnly.tar.gz: gzip compressed data, was "PippinsEyesOnly.zip", last modified: Wed Sep  3 04:58:14 2025, from Unix, original size modulo 2^32 9
87722
student@cs647:~/pippin$ gunzip PippinsEyesOnly.tar.gz
student@cs647:~/pippin$ ls
PippinsEyesOnly.tar  SaveForPippin.zip
student@cs647:~/pippin$ file PippinsEyesOnly.tar
PippinsEyesOnly.tar: Zip archive data, at least v2.0 to extract, compression method=deflate
student@cs647:~/pippin$ unzip PippinsEyesOnly.tar
Archive:  PippinsEyesOnly.tar
  inflating: PippinsEyesOnly.xz
student@cs647:~/pippin$ ls
PippinsEyesOnly.tar  PippinsEyesOnly.xz  SaveForPippin.zip
student@cs647:~/pippin$ file PippinsEyesOnly.xz
PippinsEyesOnly.xz: bzip2 compressed data, block size = 900k
student@cs647:~/pippin$ bunzip2 -k PippinsEyesOnly.xz
bunzip2: Can't guess original name for PippinsEyesOnly.xz -- using PippinsEyesOnly.xz.out
student@cs647:~/pippin$ ls
PippinsEyesOnly.tar  PippinsEyesOnly.xz  PippinsEyesOnly.xz.out  SaveForPippin.zip
student@cs647:~/pippin$ mv PippinsEyesOnly.xz.out PippinsEyesOnly.xz
student@cs647:~/pippin$ unxz PippinsEyesOnly.xz
student@cs647:~/pippin$ ls
PippinsEyesOnly  PippinsEyesOnly.tar  SaveForPippin.zip
student@cs647:~/pippin$ file PippinsEyesOnly
PippinsEyesOnly: gzip compressed data, was "PippinsEyesOnly", last modified: Wed Sep  3 04:58:11 2025, from Unix, original size modulo 2^32 2671776
student@cs647:~/pippin$ mv PippinsEyesOnly PippinsEyesOnly.gz
student@cs647:~/pippin$ gunzip PippinsEyesOnly.gz
student@cs647:~/pippin$ file PippinsEyesOnly
PippinsEyesOnly: Non-ISO extended-ASCII text, with very long lines (1568), with CRLF line terminators
```

Screenshot 1.1.2 Decompressing layers of the original zip file

The final output was an ASCII text file named PippinsEyesOnly, which was an ordinary story but concealed an OpenSSH private key. Using grep to locate the -----BEGIN OPENSSH PRIVATE KEY----- and -----END OPENSSH PRIVATE KEY----- markers the team followed with using the awk tool to extract the private key material. Utilizing the tr tool the team ensured the removal of carriage returns and or hidden characters so the key would be accepted by SSH. Then the key's permissions were changed with chmod 600, ensuring SSH's requirements that private keys be readable only by their owner.

Exploit #1	Security-Through-Obscurity SSH Key Exposure via SaveForPippin.zip

```
PippinsEyesOnly: Non-ISO extended-ASCII text, with very long lines (1568), with CRLF line terminators
student@cs647:~/pippin$ grep "BEGIN OPENSSH" PippinsEyesOnly
-----BEGIN OPENSSH PRIVATE KEY-----
student@cs647:~/pippin$ grep "END OPENSSH" PippinsEyesOnly
-----END OPENSSH PRIVATE KEY-----
student@cs647:~/pippin$ awk '/^C
student@cs647:~/pippin$ awk '/-----BEGIN .*PRIVATE KEY-----/{flag=1}flag{print}/-----END .*PRIVATE KEY-----/{flag=0}' PippinsEyesOnly> pippin_key

student@cs647:~/pippin$ tr -d '\r' <pippin_key> pippin_key.fixed && mv pippin_key.fixed pippin_key
student@cs647:~/pippin$ chmod 600 pippin_key
```

Screenshot 1.1.3 Extracting ssh key and giving it correct permissions

With the cleaned pippin key saved as pippin_key, the team was able to successfully login into the pippin account using the ssh -i option, gaining full access to the user “pippin”. Figure 1.1.4 shows the terminal session in which access to the “pippin” account was achieved.

```
student@cs647:~/pippin$ ssh -i pippin_key pippin@localhost
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-36-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

60 updates can be applied immediately.
34 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Fri Jan  5 20:43:46 2024 from 127.0.0.1
pippin@cs647:~$
```

Screenshot 1.1.4 Accessing “pippin” Account

2.2 Flag Extraction from Samba File Server

Exploit #2	Flag Extraction from Samba File Server
Description	The “pippin” account is known to store the file “pippinflag.txt” which was accessed via SSH through the exposed key from the previous attack (2.1). Utilizing the list all files (ls -a) command to unveil a hidden but unsecured Comma-Separated Values file labeled “.passwords.csv”. This file revealed a local Server Message Block (SMB) account and password which was later revealed to be a Samba File Server through a Network Mapper (nmap) scan. In conjunction with the exposed credentials and network scan, the team logged into the local smb server allowing them to locate and retrieve the flag file “pippinflag.txt”

Exploit #2	Flag Extraction from Samba File Server	
Objectives	1.2 Locate and Recover “pippinflag.txt”	
Assumptions	The “.passwords.csv” file was assumed to contain correct credentials and information. If this assumption was deemed false, more research on the Samba File Server system will be needed. This will most likely lead to an entirely different attack altogether.	
Findings	The vulnerability was located through an outdated and insecure practice of Security Through Obscurity on the “.passwords.csv” file. While attempting to hide the file through the use of the hidden files functionality of the Linux Operating system, it was easily revealed and was not secured. This allowed the team to utilize stolen credentials to log into the Samba File Server and extract the flag “pippinflag.txt”	
Mitigations	Since the primary vulnerability was present in the insecure storage of passwords the following recommendations can be made: ● Utilize a password manager ● Enable Two-Factor Authentication ● Do not store credentials in places where users have open access to it.	
Tools Used	ls -a	The list files command is utilized with -a flag to not ignore files starting with the “.” character.
	nano	This command allows the developer to edit an existing file.
	nmap	This was utilized to conduct a port scan of the local machine through the loopback address and the -A flag.
	smbclient	This command allows a system to act as a client for the SMB networking protocol, letting the user transfer files.
	get	This command is used to copy data from a remote environment to local files.

Commands Used with Syntax	`Commands in terminal:`
	`1. ls -a`
	`2. file .passwords.csv`
	`3. nano .passwords.csv`
	`4. smbclient //CS647/pippinshare/ -U pippinflag`
	`5. Enter pippinflag password from .passwords.csv “i8yO4oHbfr6i”`
	`Commands in the SMB client service:`
	`6. smb: \> get pippinflag.txt`
	`7. smb: \> exit`
	`Commands in terminal:`
	`8. nano pippinflag.txt`
	`9. whoami`	

Exploit #2	Flag Extraction from Samba File Server
Details	This exploit was directly dependent on the success of the previous exploit as all of this had to be done from inside of the “pippin” account. When initially exploring “pippin”’s filesystem, the discovery of a hidden file “.passwords.csv” revealed that there was a local smb file service. This lead to a need to further analyze the system to ensure the file the team was tasked to recover was on the system.

```
student@cs647: ~/pippin
 GNU nano 7.2                    .passwords.csv *
Account,Username,Password,Notes
bank account,pippin,password1,Need to change this password
local smb share,pippinflag,i8yO4oHbfr6i, Used to login to smb share on CS647 host
njit.edu,pip234@fake.njit.edu,12345qwertQWERT, highlander
blizzard.com,potatoPippin,secondBreakfast\!,
stackoverflow.com,pippinC0d3s,monkey, got me through my CS undergrad
paypal.com,pippin,butterflyDragon19#, never use a debit card
steampowered.com,AstroKerbyClaw,1337p4ssw0rd\!,
```

Screenshot 2.2.1 “.password.csv” contents

With that information, the team utilized the Network Mapper (nmap) tool to scan the device for active connections. The scan confirmed that the local smb share was in fact Samba, a file share server. From there, an exploration of the samba configuration file, “smb.conf”, was conducted to determine the location of this file share directory on the system.

```
pippin@cs647:~$ nmap -sV 127.0.0.1
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-09-10 14:19 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000041s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT     STATE SERVICE     VERSION
22/tcp   open  ssh         OpenSSH 9.6p1 Ubuntu 3ubuntu13.4 (Ubuntu Linux; protocol 2.0)
80/tcp   open  http        Apache httpd 2.4.58 ((Ubuntu))
139/tcp  open  netbios-ssn Samba smbd 4.6.2
445/tcp  open  netbios-ssn Samba smbd 4.6.2
631/tcp  open  ipp         CUPS 2.4
3306/tcp open  mysql       MySQL 8.0.37-0ubuntu0.24.04.1
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Screenshot 2.2.2 NMAP service scan contents

The examination of the smb.conf file led to the discovery of the share file directory “/opt/pippinshare” which contained the “pippinflag.txt” file. However, permissions were restricted to only allow users in the samba group using Samba to access it. With this information in tow, the team then moved to log into the client via the leaked credentials and recover the file.

Exploit #2	Flag Extraction from Samba File Server

```
[pippinshare]
        comment = Pippin's Share
        path = /opt/pippinshare
        read only = no
        browsable = yes
        guest ok = no
        writable = yes
        valid users = @samba
```

Screenshot 2.2.3 "/etc/samba/smb.conf" contents

The file itself was recovered utilizing the smbclient interface which was password-protected. However, the password had already been compromised due to the "pippin"'s poor attempt at hiding the passwords file. This allowed for swift and easy extraction.

```
smb: \> ls
  .                                   D        0  Sat Sep  6 16:09:59 2025
  ..                                  D        0  Sat Sep  6 16:09:59 2025
  pippinflag.txt                      N      130  Fri Sep  5 15:52:42 2025

                30267332 blocks of size 1024. 13008764 blocks available
smb: \> get pippinflag.txt
getting file \pippinflag.txt of size 130 as pippinflag.txt (42.3 KiloBytes/sec) (average 42.3 KiloBytes/sec)
smb: \> exit
pippin@cs647:~$ ls
pippinflag.txt  snap
pippin@cs647:~$ nano pippinflag.txt
pippin@cs647:~$ cat pippinflag.txt
abb712e930760451d9c4f884e6bac1f66cf77ecabaaa8a4805d2b555aadf07a1
255dfb49b7b7d647f0c0f9fe37a8a921a448b5d98f2a40451a8078b4fcda485b
pippin@cs647:~$ whoami
pippin
pippin@cs647:~$
```

Screenshot 2.2.4 Inside Samba Client and extraction of "pippingflag.txt"

3 Flags

This section will include each student's flag in a text-based format and a single screenshot showing the execution of the final exploit, printing the flag, and the result of the **`whoami`** *command.*

3.1 Percy Flores

Flag Contents
84a70c7a84b85eb40e15c457b3d736955927aec03842332b3d94b153c7973a21

7e4a24349f6310d7ca089196163b08238ec085a355304a4dc8a0e4762a82401e

Screenshot

```
pippin@cs647:~$ cat pippinflag.txt
84a70c7a84b85eb40e15c457b3d736955927aec03842332b3d94b153c7973a21
7e4a24349f6310d7ca089196163b08238ec085a355304a4dc8a0e4762a82401e
pippin@cs647:~$ whoami
pippin
pippin@cs647:~$
```

3.2 Kirlus Saad

Flag Contents

```
722b62efde5c552f5576a0853e13214c5017a45ba6bdaa71824477c1ad039f99
bdbbf3ba1f1b89922a4ce8cbf0e87c827912ef82801cf6a6eaeadfc78213da5f
```

Screenshot

```
pippin@cs647:~$ cat pippinflag.txt
722b62efde5c552f5576a0853e13214c5017a45ba6bdaa71824477c1ad039f99
bdbbf3ba1f1b89922a4ce8cbf0e87c827912ef82801cf6a6eaeadfc78213da5f
pippin@cs647:~$ whoami
pippin
```

3.3 Jake Caceres

Flag Contents

abb712e930760451d9c4f884e6bac1f66cf77ecabaaa8a4805d2b555aadf07a1

255dfb49b7b7d647f0c0f9fe37a8a921a448b5d98f2a40451a8078b4fcda485b

Screenshot

```
pippin@cs647:~$ cat pippinflag.txt
abb712e930760451d9c4f884e6bac1f66cf77ecabaaa8a4805d2b555aadf07a1
255dfb49b7b7d647f0c0f9fe37a8a921a448b5d98f2a40451a8078b4fcda485b
pippin@cs647:~$ whoami
pippin
pippin@cs647:~$
```

4 References

[1] “How Do I Create a Good Password?” *NIST*, 20 Aug. 2025, www.nist.gov/cybersecurity/how-do-i-create-good-password.

[2] *NIST Special Publication 800-63B*, pages.nist.gov/800-63-4/sp800-63b.html#appA. Accessed 10 Sept. 2025.

[3]NIST Special Publication 800-57 Part 1. *Recommendation for Key Management – Part 1: General (Rev. 5)*.https://csrc.nist.gov/publications/detail/sp/800-57-part-1/rev-5/final. Accessed 13 Sept. 2025.